✅ Comparable vs Comparator (Simple Explanation)

🔹 1. Comparable (java.lang.Comparable)

➡️ Used for default / natural sorting
➡️ Sorting logic is inside the class itself
➡️ Implemented using compareTo()

✔️ When to use

Use Comparable when a class has one natural way to sort (e.g., numbers, names, dates).

✔️ Example

```
class Student implements Comparable<Student> {
    String name;
    int age;
    Student(int age) { this.age = age; }

    @Override
    public int compareTo(Student other) {
        return this.age - other.age; // ascending
    }
}
```

Usage:

```
Collections.sort(studentList);
```

🔹 2. Comparator (java.util.Comparator)

➡️ Used for custom sorting logic
➡️ Sorting logic is outside the class (separate class, lambda, or anonymous class).
➡️ Implemented using compare()

✔️ When to use

Use Comparator when you want multiple sorting strategies, or cannot modify the class.

✔️ Example

```
Comparator<Student> byAgeDesc = (s1, s2) -> s2.age - s1.age;

Collections.sort(studentList, byAge)
```

Or another comparator:

```
Comparator<Student> byNameAsc = (s1, s2) -> s1.name.compareTo(s2.name);
```

🆚 Comparable vs Comparator (Comparison Table)

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo()	compare()
Defined In	Same class	Separate class / lambda
Sort Type	Natural order	Custom order
Multiple Sorts	❌ No	✔️ Yes
Used By	Collections.sort() / Arrays.sort()	Same but with comparator argument

---

⭐ Interview-Friendly Summary

Comparable → One default sorting logic inside the class.
Comparator → Many custom sorting logics outside the class.

📘 Comparable vs Comparator — Q&A (Interview-Ready)

✅ Q1: What is the main purpose of Comparable?

A: Comparable provides a **natural / default ordering** for a class using the compareTo() method.
Example: sorting by roll number, price, or alphabetical name.

✅ Q2: What is the main purpose of Comparator?

A: Comparator provides **custom or multiple sorting strategies** using the compare() method.
Example: sort by salary, then by name, then by age, etc.

✅ Q3: Where is the sorting logic written?

- **Comparable:** Inside the class (implements Comparable).
- **Comparator:** Outside the class (separate class, lambda, static method).

✅ Q4: Which one allows multiple sorting logics?

A: Comparator.
You can create unlimited comparators (by age, by name, by salary, etc.)

✅ Q5: Can Comparable and Comparator both be used in the same class?

A: Yes.
A class can have **one natural order** (Comparable) + **many custom orders** (Comparators).

✅ Q6: Which is better for real-world applications?

A: Comparator is generally better because it:

- avoids cluttering the model class
- supports multiple sorting rules
- works with method references and lambdas
- is more flexible

✅ Q7: Which method do they use?

- **Comparable:** compareTo(T other)
- **Comparator:** compare(T a, T b)

✅ Q8: Which package do they belong to?

- **Comparable → java.lang**
- **Comparator → java.util**

✅ Q9: Which one modifies the original class?

A: Comparable.
You must modify the class to implement Comparable.

Comparator does **not** modify the class.

✅ Q10: Which one is used by default by Collections.sort()?

A: If no comparator is passed:

Collections.sort(list); // uses Comparable

If comparator is passed:

Collections.sort(list, comparator); // uses Comparator

✅ Q11: What happens if two objects are equal?

Both compareTo() and compare() must return **0** when objects are considered equal by sorting.

✅ Q12: Should compareTo() be consistent with equals()?

Recommended, but not mandatory.
If inconsistent, objects may behave unpredictably in SortedSet or TreeMap.

✅ Q13: Which one supports method references (modern Java)?

Both, but Comparator gets the most advantage:

Comparator.comparing(Employee::getSalary)

✅ Q14: Which one should be used in Java Streams?

Comparators are preferred:

```
list.stream()
.sorted(Comparator.comparing(Employee::getName))
.toList();
```